

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 1**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class constructor with no shuffle test

The test is in testBallots.cpp with the test name

ConstructorNoShuffleTest. The Ballots constructor is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option “false”.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve all ballots within the Ballots object, notShuffleBallots, and check whether the ballots are shuffled or not.	notShuffleBallots notShuffleBallotsVector	The ballots retrieved should not be shuffled compared to notShuffleBallotsVector.	The ballots retrieved are in the same order as notShuffleBallotsVector.	None

Post condition(s) for Test:

The ballots stored in the Ballots class object, notShuffleBallots, aren't shuffled. Therefore, they match notShuffleBallotsVector. System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 2**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class constructor with shuffle test

The test is in testBallots.cpp with the test name

ConstructorShuffleTest. The Ballots constructor is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve all ballots within the Ballots object, shuffleBallots, and check whether the ballots are shuffled or not.	shuffleBallots shuffleBallotsVector	The ballots retrieved should be shuffled and therefore different from shuffleBallotsVector.	The ballots retrieved have a different order compared to shuffleBallotsVector.	None

Post condition(s) for Test:

The ballots stored in the Ballots class object, shuffleBallots, are shuffled. Therefore, they don't match shuffleBallotsVector. System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/29**Test Case ID#:** 3**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getBallot function test.

The test is in testBallots.cpp with the test name GetBallotTest.

The Ballots getBallot function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve each ballot in the Ballots object by index, notShuffleBallots, and compared with the ballot in notShuffleBallotsVector with the same index.	notShuffleBallots notShuffleBallotsVector	Each ballot by index in notShuffleBallots should be the same as the ballot by index in notShuffleBallotsVector.	The ballots retrieved by index in notShuffleBallots are the same as the ballots retrieved by index in notShuffleBallotsVector.	None
2	Test the out-of-range index when retrieving ballots from notShuffleBallots, including an index of less than zero and more than the number of ballots.	notShuffleBallots	Throw an out-of-range error.	The function throws an out-of-range error as expected on all cases.	None

Post condition(s) for Test:

Each ballot retrieved by index in notShuffleBallots matches the expected one from notShuffleBallotsVector.
System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 4**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getCandidates function test.

The test is in testBallots.cpp with the test name

GetCandidateTest. The Ballots getCandidates function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector containing the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector containing the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve candidate name vector from notShuffleBallots and compared it to notShuffleCandidatesVector	notShuffleBallots notShuffleCandidatesVector	Two vectors should be the same.	Two vectors are the same.	None
2	Retrieve candidate name vector from shuffleBallots and compared it to shuffleCandidatesVector	shuffleBallots shuffleCandidatesVector	Two vectors should be the same.	Two vectors are the same.	None

Post condition(s) for Test:

The candidate vector returned by getCandidates, regardless of whether the ballots are shuffled or not, matches the input candidate name vector.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 5**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getBallotCount function test.

The test is in testBallots.cpp with the test name

GetBallotCountTest. The Ballots getBallotCount function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve ballot count from notShuffleBallots and compared to size of notShuffleBallotsVector	notShuffleBallots notShuffleBallotsVector	The number should be the same as the size of the vector.	The two numbers are the same.	None
2	Retrieve ballot count from shuffleBallots and compared to size of shuffleBallotsVector	shuffleBallots shuffleBallotsVector	The number should be the same as the size of the vector.	The two numbers are the same.	None

Post condition(s) for Test:

The ballot count returned by getBallotCount, regardless of whether the ballots are shuffled or not, matches the input ballot vector size. System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 6**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getCandidateCount function test.

The test is in testBallots.cpp with the test name

GetCandidateCountTest. The Ballots getCandidateCount function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve candidate count from notShuffleBallots and compared to size of notShuffleCandidatesVector	notShuffleBallots notShuffleCandidatesVec tor	The number should be the same as the size of the vector.	The two numbers are the same.	None
2	Retrieve candidate count from shuffleBallots and compared to size of shuffleCandidatesVector	shuffleBallots shuffleCandidatesVector	The number should be the same as the size of the vector.	The two numbers are the same.	None

Post condition(s) for Test:

The candidate count returned by getCandidateCount, regardless of whether the ballots are shuffled or not, matches the input candidate vector size.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 7**Name(s) of Testers:** Mark Tsai**Test Description:**

Ballots class getShuffle function test.

The test is in testBallots.cpp with the test name GetShuffleTest.

The Ballots getShuffle function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A candidate vector contains the candidate names, notShuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, notShuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, notShuffleBallots, with the Ballots class constructor and shuffle option "false".

A candidate vector contains the candidate names, shuffleCandidatesVector, is created, and a ballot vector that contains the corresponding ballot information, shuffleBallotsVector, is created. These two vectors are used to create a Ballots class object, shuffleBallots, with the Ballots class constructor and shuffle option "true".

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve shuffle option from notShuffleBallots and compared to expected value "false".	notShuffleBallots	The shuffle option value should be "false".	The shuffle option value is "false".	None
2	Retrieve shuffle option from shuffleBallots and compared to expected value "true".	shuffleBallots	The shuffle option value should be "true".	The shuffle option value is "true".	None

Post condition(s) for Test:

The shuffle option value is the same as indicated when the Ballots object is created.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 8**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class assignBallot function test.

The test is in testCandidate.cpp with the test name

AssignBallotTest. The Candidate assignBallot function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object, oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assign each ballot ID in idVector to oneCandidate and push back to a temp vector to check whether the assignedBallots attribute contains those assigned IDs in the temp vector.	oneCandidate idVector	The current assigned IDs to oneCandidate should be the same as the ones in the temp vector.	The current assigned IDs to oneCandidate are the same as the ones in the temp vector.	None

Post condition(s) for Test:

The oneCandidate has the same assigned IDs from idVector in its assignedBallots attribute as the temp vector that stores the same current IDs.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 9**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class getAssignedBallots function test.
The test is in testCandidate.cpp with the test name
GetAssignBallotTest. The Candidate getAssignedBallots function
is being used.

**Indicate where are you storing the tests (what file) and the
name of the method/functions being used.**

Automated: yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.
A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object,
oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assign each ballot ID in idVector to oneCandidate and retrieve the assignedBallots attribute with getAssignedBallots to check whether it is the same as idVector.	oneCandidate idVector	The getAssignedBallots should return the same vector as idVector.	The getAssignedBallots returns the same vector as idVector.	None

Post condition(s) for Test:

The getAssignedBallots returns the assignedBallots attribute, which is the same vector as idVector.
System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 10**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class getBallotNum function test.

The test is in testCandidate.cpp with the test name

GetBallotNumTest. The Candidate getBallotNum function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object, oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assign each ballot ID in idVector to oneCandidate and retrieve number of assigned ballots with getBallotNum function.	oneCandidate idVector	The number that getBallotNum returns should be the same as the size of idVector.	Two numbers are the same.	None

Post condition(s) for Test:

The getBallotNum returns the number of ballots assigned to oneCandidate, which is the same as the size of idVector.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/10**Test Case ID#:** 11**Name(s) of Testers:** Mark Tsai**Test Description:**

Candidate class getName function test.

The test is in testCandidate.cpp with the test name GetNameTest.

The Candidate getName function is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed. A vector containing ballot IDs, idVector, is created. A string contains "Manan", name, is created. And a Candidate object, oneCandidate, is created with the name.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Retrieve the name of oneCandidate with the function getName and check whether it is the same as the name variable or not.	oneCandidate name (the string variable contains "Manan")	The retrieved string should be the same as the name variable.	Two strings are the same.	None

Post condition(s) for Test:

The getName returns the name stored in oneCandidate, which is the same as the name variable that contains "Manan". System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/11**Test Case ID#:** 12**Name(s) of Testers:** Mark Tsai**Test Description:**

The main.cpp shuffleOffFlag function test.

The test is in testMain.cpp with the test name ShuffleFlagTest.

The function in main.cpp, shuffleOffFlag, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Sets of argc and argv used in the test are created. (good_argc, good_argv) contains just the program name. (good_shuffle_argc, good_shuffle_argv) contains the program name and the correct shuffle-off flag. (bad_argc, bad_argv) contains the program name and the wrong shuffle-off flag. (many_argc, many_argv) contains the program name, the correct shuffle-off flag, and one extra argument.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Test the function with (good_argc, good_argv) and the result shuffle variable	good_argc good_argv	The result shuffle variable should be set to true, and the function returns 0.	The result shuffle variable is set to true, and the function returns 0.	None
2	Test the function with (good_shuffle_argc, good_shuffle_argv) and the result shuffle variable	good_shuffle_argc good_shuffle_argv	The result shuffle variable should be set to false, and the function returns 0.	The result shuffle variable is set to false, and the function returns 0.	None
3	Test the function with (bad_argc, bad_argv) and the result shuffle variable	bad_argc bad_argv	The result shuffle variable should be set to true, and the function returns 1.	The result shuffle variable is set to true, and the function returns 1.	None

4	Test the function with (many_argc, many_argv) and the result shuffle variable	many_argc many_argv	The result shuffle variable should be set to true, and the function returns 1.	The result shuffle variable is set to true, and the function returns 1.	None
---	---	------------------------	--	---	------

Post condition(s) for Test:

The shuffleOffFlag flag function, by default, sets the shuffle variable to true if no shuffle-off flag is taken, and sets it to false if a correct shuffle-off flag is provided. If too many arguments or the wrong flag is provided, the function returns 1 to indicate an error. Otherwise, it returns 0 to indicate a successful run.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/11**Test Case ID#:** 13**Name(s) of Testers:** Mark Tsai**Test Description:**

The main.cpp open_file function test.

The test is in testMain.cpp with the test name OpenFileTest. The function in main.cpp, open_file, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

A vector of inputs for bad file names, bad_file_name, is created. The bad inputs include wrong file name, no csv extension, and wrong directory. It contains mainly the wrong file name, but the last one is the correct one, because then the function won't be stuck in an infinite loop.

Vectors of input for good file name for plurality, plurality_file_name, and for STV, stv_file_name, are created.

A function userInput takes these vectors and simulates user input to the function by redirecting STDIN_FILENO and a function, restore_stdin_fd, to restore the STDIN_FILENO.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Redirect and simulate the user input with bad_file_name. Call open_file function. Check whether the file is opened, close the file, and restore the input redirection.	bad_file_name ../testing/stvTestMain.csv	The file is opened.	The file is opened.	The last input in bad_file_name is the correct one, so that the function won't be stuck in an infinite loop.
2	Redirect and simulate the user input with plurality_file_name. Call open_file function. Check whether the file is opened, close the file,	plurality_file_name ../testing/pluralityTestMain.csv	The file is opened.	The file is opened.	None

	and restore the input redirection.				
3	Redirect and simulate the user input with stv_file_name. Call open_file function. Check whether the file is opened, close the file, and restore the input redirection.	stv_file_name ../testing/stvTestMain.csv	The file is opened.	The file is opened.	None

Post condition(s) for Test:

The function will keep prompting the user to enter the correct file name if the wrong file name is provided. And if a correct file name is provided, the file would be opened.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/11**Test Case ID#:** 14**Name(s) of Testers:** Mark Tsai**Test Description:**

The main.cpp read_file function test.

The test is in testMain.cpp with the test name ReadFileTest. The function in main.cpp, read_file, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

Assume the ballot files do not have data errors based on the project description, no exception handling for the ballots needed.

Vectors of input for good file name for plurality, plurality_file_name, and for STV, stv_file_name, are created.

A vector, ballots_stv, containing the same ballot information as ../testing/pluralityTestMain.csv, ballots_plurality, and a vector containing the same ballot information as ../testing/stvTestMain.csv.

A function userInput takes these vectors and simulates user input to the function by redirecting STDIN_FILENO and a function, restore_stdin_fd, to restore the STDIN_FILENO.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Redirect and simulate the user input with plurality_file_name. Call open_file function. Call read_file function and store the result Ballots object. Close the file, and restore the input redirection. Check whether the Ballots object contains all ballots as expected.	plurality_file_name ballots_plurality ../testing/pluralityTestMain.csv	The Ballots object returned by the function should contain the same ballots as ballots_plurality.	The Ballots object returned by the function contains the same ballots as ballots_plurality.	None

2	Redirect and simulate the user input with stv_file_name. Call open_file function. Call read_file function and store the result Ballots object. Close the file, and restore the input redirection. Check whether the Ballots object contains all ballots as expected.	stv_file_name ballots_stv ../testing/stvTestMain.csv	The Ballots object returned by the function should contain the same ballots as ballots_stv.	The Ballots object returned by the function contains the same ballots as ballots_stv.	None
---	--	--	---	---	------

Post condition(s) for Test:

The function returns the Ballots object with the same ballot information as stored in the opened file, regardless of whether it is for plurality or STV.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 15**Name(s) of Testers:** Mark Tsai**Test Description:**

The main.cpp prompt_user_seatNum function test.

The test is in testMain.cpp with the test name SeatNumTest. The function in main.cpp, prompt_user_seatNum, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

A function userInput takes these vectors and simulates user input to the function by redirecting STDIN_FILENO and a function, restore_stdin_fd, to restore the STDIN_FILENO.

A vector of input for bad inputs, bad_seatNum, is created. It includes numbers outside of the integer range, alphabets, 0, and non-positive integers. But the last one is the correct one, because then the function won't be stuck in an infinite loop. A vector of input for good input, good_seatNum, is created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Redirect and simulate the user input with bad_seatNum. Call prompt_user_seatNum function. Restore the input redirection. Check the final variable value.	bad_seatNum	The final variable should be set to 5.	The final variable is set to 5.	The last input in bad_seatNum is the correct one, so that the function won't be stuck in an infinite loop.
2	Redirect and simulate the user input with good_seatNum. Call prompt_user_seatNum function. Restore the input redirection. Check the final variable value.	good_seatNum	The final variable should be set to 10.	The final variable is set to 10.	None

Post condition(s) for Test:

The function will keep prompting the user to enter a valid seat number if the wrong one is provided. The function takes a positive integer and sets the final variable to that value.

System state remains unchanged.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 16**Name(s) of Testers:** Mark Tsai**Test Description:**

The main.cpp prompt_user_alg function test.

The test is in testMain.cpp with the test name AlgTest. The function in main.cpp, prompt_user_alg, is being used.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

Assume the user input the correct option for algorithm type corresponding to the input ballot file type based on what Shana said. A function userInput takes these vectors and simulates user input to the function by redirecting STDIN_FILENO and a function, restore_stdin_fd, to restore the STDIN_FILENO.

A vector for bad inputs, bad_alg, is created. It includes numbers outside of the integer range, alphabets, 0, a negative number, and a positive number that is not 1 or 2 (valid options). But the last one is the correct one, because then the function won't be stuck in an infinite loop. A vector for option one input, good_alg_one, and a vector for option two input, good_alg_two, are created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Redirect and simulate the user input with bad_alg. Call prompt_user_alg function. Restore the input redirection. Check the final variable algorithm type as a string.	bad_alg	The final algorithm type as a string should be "Plurality".	The final algorithm type is "Plurality".	The last input in bad_alg is the correct one, so that the function won't be stuck in an infinite loop.
2	Redirect and simulate the user input with good_alg_one. Call prompt_user_alg function. Restore the input redirection. Check the final variable	good_alg_one	The final algorithm type as a string should be "Plurality".	The final algorithm type is "Plurality".	None

	algorithm type as a string.				
3	Redirect and simulate the user input with good_alg_two. Call prompt_user_alg function. Restore the input redirection. Check the final variable algorithm type as a string.	good_alg_two	The final algorithm type as a string should be "STV".	The final algorithm type is "STV".	None

Post condition(s) for Test:

The function will keep prompting the user to enter a valid algorithm option if the wrong one is provided. The function takes either 1 or 2 and sets the final variable to the corresponding option as a string. 1 represents the Plurality and 2 represents the STV. System state remains unchanged.

Project Name: Project 1: Voting System

Team# 11

Test Stage: Unit

Test Date: 2025/03/27

Test Case ID#: 17

Name(s) of Testers: Andy Dang

Test Description:

Plurality constructor test. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes

Results: Pass

Preconditions for Test:

The seats and ballots arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create an instance of the Plurality class	ballots_plurality, seats = 2	The member attributes should be initialized correctly.	The member attributes are initialized correctly.	None

Post condition(s) for Test:

An instance of the Plurality class has been created. The state of the system has not been changed.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 18**Name(s) of Testers:** Andy Dang**Test Description:**

Runs a simple plurality election. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.**Automated:** yes**Results:** Pass**Preconditions for Test:**

The Plurality constructor works properly, and the member attributes are initialized accordingly

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Runs a plurality election	ballots_plurality, seats = 2	Bill and Alice should win the election	Bill and Alice won the election	None
2	Runs a plurality election with one candidate	ballots_plurality_four, seats = 3	Andy should win the election	Andy won the election	None
3	Runs a plurality election with one seat	ballots_plurality, seats = 1	Bill should win the election	Bill won the election	None
4	Runs a plurality election with one candidate and one seat	ballots_plurality_four, seats = 1	Andy should win the election	Andy won the election	None

Post condition(s) for Test:

A plurality election has been processed, with Bill and Alice being winners. The system is now in a state where an election has been processed.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 19**Name(s) of Testers:** Andy Dang**Test Description:**

Test for the output of a plurality election. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

The Plurality constructor works properly, and the member attributes are initialized accordingly

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create an instance of the Plurality class	ballots_plurality, seats = 2	Election type, # of seats, # of ballots, # of candidates, winners, and losers should be displayed, with each candidate having the correct percentage of ballots.	vElection type, # of seats, # of ballots, # of candidates, winners, and losers are displayed, with each candidate having the correct percentage of ballots.	The function that is tested, displayElectionDetails(), is called inside runElection()

Post condition(s) for Test:

A plurality election has been processed, and the results are displayed on the terminal. The system is now in a state where an election has been processed.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 20**Name(s) of Testers:** Andy Dang**Test Description:**

Time the plurality election with 100,000 ballots. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes**Results:** Pass**Preconditions for Test:**

None

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Time the process of creating ballots, creating a Plurality instance, and running an election	ballots_plurality_two, candidates_two seats = 5	The time it takes to process the election should be under 5 minutes.	The time it takes to process the election is under 5 minutes.	None

Post condition(s) for Test:

A plurality election has been processed in under 5 minutes. The state of the system has not been changed.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/27**Test Case ID#:** 21**Name(s) of Testers:** Andy Dang**Test Description:**

Tests the randomizer if there are multiple winning candidates for one seat. Located in testPlurality.cpp.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes / no**Results:** Pass**Preconditions for Test:**

The seats and ballots arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run a plurality election with a ballot that has 2 candidates tying for the final seat	ballots_plurality_three, seats = 3	Bill and Alice should be winners, and Siyang and Preeti should be losers	Bill and Alice are winners, and Siyang and Preeti are losers	None
2	Repeat Step 1 multiple times (at least 10)	ballots_plurality_three, seats = 3	Ahmed should win around half the time, and Sally should win around half the time	Ahmed wins around half the time, and Sally wins around half the time	Run ./testPlurality multiple times to test the randomizer (tiebreaker)
3	Run the same plurality election multiple times (at least 10) that has more than 2 candidates tying for the final seat	ballots_plurality_seven, seats = 1	Manan should win $\frac{1}{3}$ of the time, Mark should win $\frac{1}{3}$ of the time, Michael should win $\frac{1}{3}$ of the time	Manan won 5 times, Mark won 4 times, Michael won once	This was done in the same way as Step 2
4	Run the same plurality election multiple times (at least 10) that has everyone tying for the final seat	ballots_plurality_eight, seats = 1	Each of the candidates is expected to win once	4 candidates won once, 2 won twice, and 2 never won	This was done in the same way as Step 2

Post condition(s) for Test:

Multiple plurality elections have been processed, with Bill and Alice being winners, and Siyang and Preeti losers. Ahmed is expected to win half the time, and Sally is expected to win half the time. The system is now in a state where an election has been processed.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 22**Name(s) of Testers:** Michael Dunn**Test Description:**

Multiple candidates are directly elected by the STV election without needing winner or loser vote redistribution.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run an STV election with a ballot that has 2 candidates meeting the Droop quota and being selected as winners.	basic_win, seats = 2.	Bill Jones and Sally Ride should be winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee should be losers. Both lists should be returned in that order.	Bill Jones and Sally Ride were selected as winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee were selected as losers.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 23**Name(s) of Testers:** Michael Dunn**Test Description:**

Testing that when a winner hits the Droop quota and is moved to the winners' list, any additional ballots they receive must immediately be redistributed to the next available option.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run an STV election, and when distributing a ballot that would've gone to a candidate already in the winners' list, assign it instead the ballot's next option, causing another candidate to reach the Droop quota.	winner_reassign_win, seats = 2.	Bill Jones and Sally Ride should be winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee should be losers. Both lists should be returned in that order.	Bill Jones and Sally Ride were selected as winners, and Alice Mix, Ahmed Mohamed, Siyang Xiong, and Preeti Banerjee were selected as losers.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome. The ballot that was reassigned is assigned to one candidate and therefore only shows up once.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 24**Name(s) of Testers:** Michael Dunn**Test Description:**

Testing the case that all ballots are distributed and there are still seats that need to be filled. The candidate with the fewest (non-zero) votes has their votes redistributed, which causes a candidate that hasn't 'lost' yet to hit the Droop quota, and the remaining seat being filled.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Run an STV election and distribute all votes to their respective candidates. After votes are distributed, find the candidate with the lowest votes and redistribute them, which elects the final candidate.	loser_reassign_win, seats = 2.	Bill Jones will hit the Droop quota and be selected as a winner, and Sally Ride will receive a redistributed ballot from Siyang Xiong, allowing them to hit the Droop quota.	Bill Jones and Sally Ride were selected as winners, and Alice Mix, Ahmed Mohamed, Preeti Banerjee, and Siyang Xiong, were selected as losers. Siyang Xiong's redistributed ballot was successfully removed from their assigned ballots list and reassigned to Sally Ride.	

Postcondition(s) for Test:

Election details and audit files match the expected election outcome. The ballot that was reassigned is assigned to one candidate and therefore only shows up once.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System

Team# 11

Test Stage: Unit

Test Date: 2025/03/28

Test Case ID#: 25

Name(s) of Testers: Michael Dunn

Test Description:

Testing the display of election details after an election is run.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes

Results: Pass

Preconditions for Test:

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Finish an STV election and ensure that the correct details are output to the screen.	basic_win, seats = 2.	Election type: STV Number of seats: 2 Number of ballots: 6 Number of candidates: 6 Winners: Bill Jones Sally Ride Losers: Alice Mix Ahmed Mohamed Siyang Xiong Preeti Banerjee	Election type: STV Number of seats: 2 Number of ballots: 6 Number of candidates: 6 Winners: Bill Jones Sally Ride Losers: Alice Mix Ahmed Mohamed Siyang Xiong Preeti Banerjee	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** Unit**Test Date:** 2025/03/28**Test Case ID#:** 26**Name(s) of Testers:** Michael Dunn

Test Description: In the case that two candidates receive the same number of votes, and there is only one seat to be elected, the STV election will need to choose between the two candidates randomly. This test ensures that only the two candidates with the same number of votes are considered for the seat.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

The seats, ballots, and audit filename arguments exist and are valid.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	After the ballots are distributed to their first choice candidate in an STV election, there is a tie between two candidates. One of the tied candidates and their votes will be redistributed, leading to the other candidate to win.	basic_win, seats = 1.	Sally Ride or Bill Jones hit the Droop quota and wins the election.	The losers are Alice Mix, Ahmed Mohamed, Siyang Xiong, Preeti Banerjee, and either Bill Jones or Sally Ride are selected as losers in that order.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 27**Name(s) of Testers:** Manan Chaturvedi

Test Description: Testing that the ballots are shuffled within a 90% minimum rate of difference between the ballots from a given .csv file versus ballots shuffled from the same .csv file. This utilized the Ballots.cpp file along with main.cpp.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

No existing preconditions.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	A .csv file is loaded in automatically and a ballot class is made.	STV3.csv (particular file) Ballot ballots; ifstream file;	The file is valid and can be read.	The actual result is such the case.	
2	The same file is read and shuffled, then compared with the original file. Repeat 1000 times.	Ballots shuffled; int shuffleValid = 1000; int not_equal = 0; ifstream file2;	int not_equal >= 90; Increases by 1 for each shuffled ballot, a total of 100 ballots.	Varies from ~90 to 100.	This is set up for a 100 ballot system, however could be adjusted for more - though small values are better for proof.

Postcondition(s) for Test:

The correct election details are printed on the screen.

Shows number of ballots mismatched for each scenario on the screen.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 28**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests to ensure that the system's STV election process does not result in a different result whether or not the ballot is shuffled. Uses STV.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

There exists a valid .csv file for a STV algorithm, along with valid user input for STV algorithm and seat value and audit file.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Load in a .csv file for STV with valid user input - shuffle a second set of ballots from the same file.	int seatNum Ballot ballots; Ballot shuffled; ifstream audit_file STV STVelection; STV STVelectionS ifstream file;	Valid path for .csv file, positive seatNum, and valid audit_file path.	Valid paths and values are checked throughout the test - such is true.	
2	Run 2 STV elections and compare the results of each with the winners sorted by # of assigned ballots.	vector<Candidate> regWinners = STVelection->getWinners(); vector<Candidate> shfWinners = STVelectionS->getWinners(); vector<int> regWin(regWinners.size()); vector<int> shfWin(shfWinners.size());	regWin is exactly the same as shfWin.	regWin and shfWin are the same, even in the event of ties.	

Postcondition(s) for Test:

The correct election details are printed on the screen.
Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 29**Name(s) of Testers:** Manan Chaturvedi

Test Description: Tests to ensure that the system's Plurality election process does not result in a different result whether or not the ballot is shuffled. Uses Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

There exists a valid .csv file for a Plurality algorithm, along with valid user input for Plurality algorithm and seat value

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Load in a .csv file for Plurality with valid user input - shuffle a second set of ballots from the same file.	int seatNum Ballot ballots; Ballot shuffled; Plurality pluralityElection; Plurality pluralityElectionS; ifstream file;	Valid path for .csv file, and positive seatNum.	Valid paths and values are checked throughout the test - such is true.	
	Run 2 STV elections and compare the results of each with the winners sorted by # of assigned ballots.	vector<Candidate> regWinners = Pluralityelection->getWinners(); vector<Candidate> shfWinners = pluralityElectionS->getWinners(); vector<int> regWin(regWinners.size()); vector<int> shfWin(shfWinners.size());	regWin is exactly the same as shfWin.	regWin and shfWin are the same, even in the event of ties.	

Postcondition(s) for Test:

The correct election details are printed on the screen.
Election details match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 30**Name(s) of Testers:** Michael Dunn

Test Description: Tests with the most minimal scenario involving 1 candidate with 1 vote in both a Plurality and STV election. This involves using the entire system - STV.cpp, Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp automated testing file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

Valid .csv files for STV and Plurality, with seatNum = 1 and a valid audit path for STV and a valid algorithm input for STV and Plurality.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Load in 2 .csv files for STV and Plurality and set the number of seats to 1 - then run each election to get the winners. The size should be 1 for each and the number of ballots should be set to the total number of ballots for plurality/the droop quota for STV.	int seatNum = 1; Ballot ballots; Ballot ballotp; ifstream file; ifstream file2;	There should be a winner size of one, losers should be a size of 0, and there should be 1 vote for each scenario.	winner.size() = 1 loser.size() = 0 Ballot number is set to 1 in both cases, but meets droop quota for STV.	

Postcondition(s) for Test:

The correct election details are printed on the screen

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 31**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test just runs a basic Plurality election, but confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Uses Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

There exists a .csv file with an input of the total number of seats and a valid input for a Plurality algorithm.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The Plurality election runs with a set number of seats, and then performs runElection to get the resulting number of winners and losers.	int seatNum; Plurality pluralityElection; ifstream file; Ballots ballot;	Should have a set number of seats, a set number of winners and losers, along with a Ballot class and valid csv file.	Such is the case for each scenario that there is a valid value for each variable.	
2	Then loops through each winner and loser to confirm their position in their list.	vector<Candidate> winners; vector<Candidate> losers; int loser_size = pluralityElection->getCandidates().size() - seatNum;	Each winner should have a .getBallotNum value greater than in losers.	Such is the case for each scenario that there is a valid value for each variable.	

Postcondition(s) for Test:

The correct election details are printed on the screen.
Election details match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/28**Test Case ID#:** 32**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test just runs a basic Plurality election, however there is a tie. This confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Uses Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

There exists a .csv file with an input of the total number of seats and a valid input for a Plurality algorithm.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The Plurality election runs with a set number of seats, and then performs runElection to get the resulting number of winners and losers.	int seatNum; Plurality pluralityElection; ifstream file; Ballots ballot;	Should have a set number of seats, a set number of winners and losers, along with a Ballot class and valid csv file.	Such is the case for each scenario that there is a valid value for each variable.	
2	Then loops through each winner and loser to confirm their position in their list.	vector<Candidate> winners; vector<Candidate> losers; int loser_size = pluralityElection->getCandidates().size() - seatNum;	Each winner should have a .getBallotNum value greater than OR EQUAL in losers.	Such is the case for each scenario that there is a valid value for each variable.	
3	Check for valid audit file location and access.	bool access(audit_file);	Should be true.	Returns true for the audit_file.	

Postcondition(s) for Test:

The correct election details are printed on the screen.
Election details match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 33**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test just runs a basic STV election, but confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Uses STV.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

Valid .csv files for STV and Plurality, with seatNum = 1 and a valid audit path for STV.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The STV election runs with a set number of seats, then performs runElection to get the resulting number of winners and losers.	int seatNum; STV STVelection; ifstream file; ifstream audit_file; Ballots ballot;	Should have a set number of seats, a set number of winners and losers, along with a Ballot class and valid .csv and audit file.	Such is the case for each scenario that there is a valid value for each variable.	
2	Then loops through each winner and loser to confirm their position in their list.	vector<Candidate> winners; vector<Candidate> losers; int loser_size = STVelection->getCandidates().size() - seatNum;	Each winner should have a .getBallotNum value greater than in losers.	Such is the case for each scenario that there is a valid value for each variable.	
3	Check for valid audit file location and access.	bool access(audit_file);	Should be true.	Returns true for the aud-t_file.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/28**Test Case ID#:** 34**Name(s) of Testers:**

Test Description: This test just runs a basic STV election, however there is a tie. This confirms that the winners and losers are accurate to the number of seats and total seats along with each winner having more votes than all of the losers. Uses STV.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

Valid .csv files for STV and Plurality, with seatNum = 1 and a valid audit path for STV.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	The STV election runs with a set number of seats, then performs runElection to get the resulting number of winners and losers.	int seatNum; STV STVelection; ifstream file; ifstream audit_file; Ballots ballot;	Should have a set number of seats, a set number of winners and losers, along with a Ballot class and valid .csv and audit file.	Such is the case for each scenario that there is a valid value for each variable.	
2	Then loops through each winner and loser to confirm their position in their list.	vector<Candidate> winners; vector<Candidate> losers; int loser_size = STVelection->getCandidates().size() - seatNum;	Each winner should have a .getBallotNum value greater than OR EQUAL in losers.	Such is the case for each scenario that there is a valid value for each variable.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 35**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test determines the overall time that it takes for a STV election to perform a valid and verifiable election within a given amount of time. Uses STV.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

Valid .csv files for STV and Plurality, with seatNum = 1 and a valid audit path for STV.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Starts a timer, but then performs a STV election as in previous tests - also checks validity using previous methods.	t0 = chrono::high_resolution::clock::now(); int seatNum; STV STVelection; ifstream file; ifstream audit_file; Ballots ballot;	Should have a set number of seats, a set number of winners and losers, along with a Ballot class and valid .csv and audit file - also should have a started timer.	Such is the case for each scenario that there is a valid value for each variable.	
2	Validates the winners, losers, and the audit_file and stops the timer to record if less than 5 minutes.	t1 = chrono::high_resolution::clock::now(); vector<Candidate> winners; vector<Candidate> losers; int loser_size = STVelection->getCandidates().size() - seatNum;	Should have a valid number of winners, losers, audit_file, and a valid time under 5 minutes.	Such is the case for each scenario that there is a valid value for each variable.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 36**Name(s) of Testers:** Manan Chaturvedi

Test Description: This test determines the overall time that it takes for a Plurality election to perform a valid and verifiable election within a given amount of time. Uses Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

There exists a .csv file with an input of the total number of seats and a valid input for a Plurality algorithm.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Starts a timer, but then performs a Plurality election as in previous tests - also checks validity using previous methods.	t0 = chrono::high_resolution::clock::now(); int seatNum; Plurality pluralityElection; ifstream file; Ballots ballot;	Should have a set number of seats, a set number of winners and losers, along with a Ballot class and valid .csv and audit file - also should have a started timer.	Such is the case for each scenario that there is a valid value for each variable.	
2	Validates the winners, losers and stops the timer to record if less than 5 minutes.	t1 = chrono::high_resolution::clock::now(); vector<Candidate> winners; vector<Candidate> losers; int loser_size = pluralityElection->getCandidates().size() - seatNum;	Should have a valid number of winners, losers, and a valid time under 5 minutes.	Such is the case for each scenario that there is a valid value for each variable.	

Postcondition(s) for Test:

The correct election details are printed on the screen.
Election details match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 37**Name(s) of Testers:** Manan Chaturvedi

Test Description: This case handles the event that there is a large number of ballots with a number of seats where the number of seats are larger than the total number of candidates. This involves using the entire system - STV.cpp, Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp automated testing file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Pass**Preconditions for Test:**

Valid .csv files for STV and Plurality, with seatNum = 1 and a valid audit path for STV and a valid algorithm input for STV and Plurality.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Starts a STV and Plurality election, but records a number of seats higher than the number of candidates and runs each election.	int seatNum; Plurality pluralityElection; STV STVelection; ifstream file; ifstream audit_file; Ballots ballots; Ballots ballots2; int winners_size = STVelection /pluralityElection->getWinners().size(); int losers_size = pluralityElection/ STVelection->getLosers().size();	Should have all of the candidates in winners_size while losers_size should have 0 candidates in both cases.	Such is the case for each election that there is nobody in the losers list but everyone is in the winners list.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: Project 1: Voting System**Team# 11****Test Stage:** System**Test Date:** 2025/03/29**Test Case ID#:** 38**Name(s) of Testers:** Manan Chaturvedi

Test Description: This case handles the event that there is a small number of ballots with a number of seats where the number of seats are larger than the total number of candidates. This involves using the entire system - STV.cpp, Plurality.cpp, Ballots.cpp, Candidate.cpp, and main.cpp within the testSystem.cpp automated testing file.

Indicate where you are storing the tests (what file) and the name of the method/functions being used.

Automated: Yes**Results:** Fail**Preconditions for Test:**

Valid .csv files for STV and Plurality, with seatNum = 1 and a valid audit path for STV and a valid algorithm input for STV and Plurality.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Starts a STV and Plurality election, but records a number of seats higher than the number of candidates and runs each election.	int seatNum; Plurality pluralityElection; STV STVelection; ifstream file; ifstream audit_file; Ballots ballots; Ballots ballots2; int winners_size = STVelection / pluralityElection->getWinners().size(); int losers_size = pluralityElection / STVelection->getLosers().size();	Should have just one of the candidates in winners_size while losers_size should have the rest of the candidates in both cases.	Oddity: Everyone is in the plurality election's winner list, while in the STV election only one person is in the winner list, but everyone - including the winner - is in the losers list.	

Postcondition(s) for Test:

The correct election details are printed on the screen.

Election details and audit files match the expected election outcome.

Project Name: The project #, name of your system, and the team#

Test Stage: Indicate whether it is a unit test or a system test.

Test Date: The date the test was performed.

Test Case ID#: A unique ID is required. Decide on a naming convention and use numbering. Example: Ballot_Shuffle_1

Name(s) of Testers: List the names of anyone involved in running this test case.

Test Description: Describe briefly the test objective.

Automated: Indicate if the test is completely automated or being checked manually. (If you have methods running the tests and checking results, select “yes”. If you are manually checking results, indicate manual by selecting the “no.”)

Results: Indicate if the test passed or failed.

Step #: You will be listing the test steps in order. This number is the step number in the process.

Test Step Description: Details of the test step.

Test Data: What the test data will be for this step. Be clear on what the input data will be. If using a specific file, be clear on the name.

Expected Result: What result are you expecting from the program component or system.

Actual Result: What result were returned based on the test.

Post condition for Test: What will be true after the test has been run? Has the state of the system changed in any way?

Notes: Comments and notes for you and your team members.